

Cascading Style Sheets

Что такое CSS?

Основным понятием CSS (Cascading Style Sheets – каскадные таблицы стилей) является **стиль** – то есть набор правил оформления и форматирования, который может быть применен к различным элементам страницы.

В стандартном HTML для присвоения какому-либо элементу определенных свойств (таких, как цвет, размер, положение на странице и пр.) приходится каждый раз описывать эти свойства, даже если на одной странице должны располагаться 10 или 110 таких элементов, неотличающихся друг от друга.

Что такое CSS?

CSS действует другим, более удобным и экономичным способом. Для присвоения какому-либо элементу определенных характеристик необходимо один раз описать его свойства и определить это описание как стиль. А в дальнейшем просто указывать, что элемент, который нужно оформить соответствующим образом, относится к данному стилю.

Описание стилей

Описание стиля состоит из двух частей: **селектора** и **блока определений**, заключенного в фигурные скобки .

Каждое определение заканчивается точкой с запятой. Определение включает свойство и значение этого свойства, разделенные двоеточием.



Правила оформления CSS-кода

Синтаксис

- Пробел перед фигурной скобкой.
- Для отступов внутри правил используйте два пробела.
- Каждое свойство с новой строки.
- Пробел после двоеточия в определении свойства
- Закрывающая фигурная скобка на новой строке.
- В конце строки с определением стиля должна стоять точка с запятой.

Правила оформления CSS-кода

Синтаксис

- Значение цветов не сокращается.
- Название классов, тегов и свойств пишется строчными буквами.
- Нули не пропускаются (0.5, а не .5).
- Используйте двойные кавычки.
- Пробелы после запятой в цветах (`rgba(0, 0, 0, 0.5)`), а не `rgba(0,0,0,0.5)`).
- Пробел до и после комбинатора (`p > a`).

Правила оформления CSS-кода

Имена классов

- Имена классов пишутся строчными буквами, между несколькими словами используется **дефис** (но не **snake_case** или **camelCase**).
- **Дефисы** служат разделителями во взаимосвязанных классах (например, **.button** и **.button-cancel**).
- Имена должны быть такими, чтобы по ним можно было быстро определить, какому элементу на странице задан класс: избегайте сокращений, но не делайте их слишком длинными.
- Для именования классов используются **английские слова и термины**. **Не используйте транслит** для названия классов и атрибутов.

Правила оформления CSS-кода

Задание 1. Отформатировать следующий CSS-код в соответствии с правилами оформления.

```
.menuItem{text-align:center}
#menu ul{
    padding-left : 0; margin:.5%
}
#menu li {
display : inline-block;
width :16%;
background: #F00;
}
#menu a {
text-decoration:none;
color:white;}
.PICT{
width: 70%;
background : rgba(0,255,0,.5);
margin :5px auto; padding:5px    0}
```


Связь между CSS и HTML

Каждому атрибуту и тегу HTML, отвечающему за форматирование, можно поставить в соответствие некоторое свойство стилей CSS.

Это позволяет «очистить» код от множества тегов и атрибутов форматирования.

Ниже приведены некоторые соответствия между атрибутами (тегами) и стилями:

<code></code>	<code>color: red;</code>
<code></code>	<code>font-size: 16px;</code>
<code>...</code>	<code>font-weight: bold;</code>
<code>align="center"</code>	<code>text-align: center;</code>

Связь между CSS и HTML, пример

```
<p align="justify">  
  <font size="3" color="blue">  
    <b>  
      Форматирование текста.  
    </b>  
  </font>  
</p>
```



CSS

```
p {  
  text-align: justify;  
  font-weight: bold;  
  font-size: 16px;  
  color: blue  
}
```

HTML

```
<p>  
  Форматирование текста.  
</p>
```

Типы селекторов

Селектор – это формальное описание того элемента или группы элементов, к которым применяется описание стиля.

В качестве селекторов могут выступать:

1. **Название HTML-тега** (h1, p, td, а и т. д.). В этом случае заданные в блоке определений стили будут применены ко всем HTML-тегам, использованным в качестве селектора.

Код CSS

```
p {  
  text-align: right;  
  font-size: 12px;  
}
```

Код HTML

```
<p> стиль </p>  
<p> описание </p>
```

Типы селекторов

В качестве селекторов могут выступать:

2. **Имена классов**. CSS позволяет присваивать стили не всем элементам, созданным с помощью некоторого тега, а избирательно.

При описании стиля CSS для класса перед его названием ставится точка (.)

Код CSS

```
.center {  
  text-align: center;  
  font-weight: bold;  
}
```

Код HTML

```
<h1 class="center"> CSS </h1>  
<p class="center"> стиль </p>  
<p> описание </p>
```

Типы селекторов

В качестве селекторов могут выступать:

3. **Идентификаторы элементов.** CSS позволяет описать стиль отдельного элемента. Для этого в HTML-коде этому элементу необходимо присвоить идентификатор с помощью атрибута **id**.

При описании CSS стиля перед идентификатором элемента указывается решетка (#).

Код CSS

```
#par {
  color: red;
  text-align: center;
}
```

Код HTML

```
<p id="par"> CSS </p>
<p> стиль </p>
```

Типы селекторов

Задание 2. Очистите следующий HTML-код (написать CSS стили и HTML-код).

Код HTML

```
<h1 align="center">
  <font color="maroon">
    <ins>Основы HTML</ins>
  </font>
</h1>
<h2 align="center">
  <font color="maroon">
    <i>Оглавление</i>
  </font>
</h2>
```

Соответствие:

	color: red
<i>...</i>	font-style: italic
<ins>...</ins>	text-decoration: underline
align="center"	text-align: center

Каскадность CSS

Каскадность — возможность языка CSS "перекрывать" свойства друг на друга, а также расширять свойства элементов в селекторах.

Каскадность CSS

Расширение свойств в селекторах

Для одного элемента страницы могут использоваться несколько селекторов, в результате все CSS свойства элемента "суммируются".

Код CSS

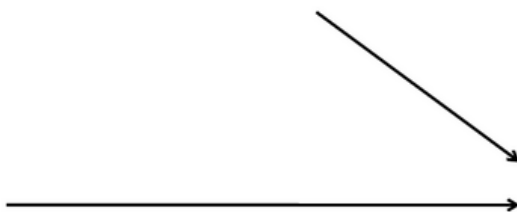
```
p {  
  color: blue;  
}  
#bold {  
  font-weight: bold;  
}  
.center {  
  text-align: center;  
}
```

Код HTML

```
<p id="bold" class="center"> стиль </p>
```

Код CSS

```
{  
  color: blue;  
  font-weight: bold;  
  text-align: center;  
}
```



Каскадность CSS

Приоритет селекторов

Для одного элемента страницы могут использоваться несколько селекторов.

Если их свойства "перекрываются", тогда они применяются в следующем порядке:

1. селектор по идентификатору;
2. селектор по классу;
3. селектор по тегу.

Каскадность CSS

Пример. Применим все типы селекторов к одному элементу.

Код CSS

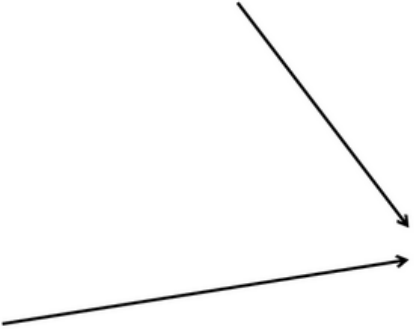
```
.center {  
  text-align: center;  
}  
p {  
  color: blue;  
}  
#par {  
  color: red;  
  text-align: right;  
}
```

Код HTML

```
<p id="par" class="center"> стиль </p>
```

Код CSS

```
{  
  color: red;  
  text-align: right;  
}
```



Каскадность CSS

Переопределение свойств в классах

Один и тот же элемент может относиться к нескольким классам, тогда в атрибуте **class** они записываются через пробел:

```
<тег class="class-1 class-2">
```

При этом, если в обоих классах определены одинаковые стили, то к элементу применяются стили того класса, который в CSS определен последним.

При этом порядок перечисления классов в атрибуте **class** не имеет значения.

Каскадность CSS

Пример. Применим стили двух классов к одному элементу в разном порядке.

Код CSS

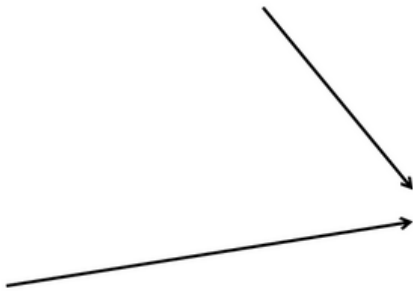
```
.right-red {  
  color: red;  
  text-align: right;  
}  
.center-blue {  
  color: blue;  
  text-align: center;  
}
```

Код HTML

```
<p class="right-red center-blue"> стиль_1 </p>  
<p class="center-blue right-red"> стиль_2 </p>
```

Код CSS

```
{  
  color: blue;  
  text-align: center;  
}
```



Комбинация селекторов

Группировка селекторов

Если несколько селекторов имеют одинаковое описание, их можно указать **через запятую** перед описанием стиля.

Код CSS

```
#par, h2 {  
    color: red;  
    text-align: center;  
}
```

Комбинация селекторов

Контекстные селекторы

Контекстные селекторы позволяют описывать стили для тегов, вложенных друг в друга. В этом случае селекторы **разделяются пробелом**.

Код CSS

```
li a {  
    color: green;  
}
```


Комбинация селекторов

Соединение селекторов

CSS позволяет соединять селекторы по тегу и селекторы по классу. Для этого после названия тега записывают точку и название класса без пробелов

Код CSS

```
.color-text {  
  color: red;  
  text-align: center;  
}  
p {  
  color: blue;  
}
```



Код CSS

```
.color-text {  
  color: red;  
  text-align: center;  
}  
p.color-text {  
  color: blue;  
}
```

Комбинация селекторов

Приоритет селекторов

Для одного элемента страницы могут использоваться несколько селекторов. Если их свойства "перекрываются", тогда они применяются в следующем порядке:

1. селектор по идентификатору;
2. селектор по классу, соединенный с тегом;
3. селектор по классу (если классов у элемента несколько, то приоритет выше у того класса, который в CSS описан позже);
4. селектор по тегу.

Комбинация селекторов

Пример. Определите стиль каждого слова

Код HTML

```
<p class="big-font bold-text"> Один </p>
<ul>
  <li class="bold-text big-font"> Два </li>
  <li class="big-font"> Три </li>
  <li class="bold-text"> Четыре </li>
  <li> Пять </li>
</ul>
```

Код CSS

```
.bold-text {
  font-weight: bold;
  color: green;
}
.big-font {
  color: brown;
  font-size: 30px;
}
p {
  font-size: 40px;
}
p.bold-text {
  color: blue;
}
ul {
  font-size: 14px;
  color: red;
}
```

Комбинация селекторов

Пример. Определите стиль каждого слова

Код HTML

```
<p class="big-font bold-text"> Один </p>
<ul>
  <li class="bold-text big-font"> Два </li>
  <li class="big-font"> Три </li>
  <li class="bold-text"> Четыре </li>
  <li> Пять </li>
</ul>
```

Результат:

Один

- Два
- Три
- Четыре
- Пять

Код CSS

```
.bold-text {
  font-weight: bold;
  color: green;
}
.big-font {
  color: brown;
  font-size: 30px;
}
p {
  font-size: 40px;
}
p.bold-text {
  color: blue;
}
ul {
  font-size: 14px;
  color: red;
}
```

Комбинация селекторов

Приоритет селекторов

Для одного элемента страницы могут использоваться несколько селекторов. Если их свойства "перекрываются", тогда они применяются в следующем порядке:

1. селектор по идентификатору;
2. селектор по классу, соединенный с тегом;
3. селектор по классу (если классов у элемента несколько, то приоритет выше у того класса, который в CSS описан позже);
4. селектор по тегу.

Комбинация селекторов

Задание 3. Применить стили к html-коду, чтобы страница соответствовала результату.

Код CSS

```
.blue-bold {  
  font-weight: bold;  
  color: blue;  
}  
  
.big-font {  
  font-size: 20px;  
  color: green;  
}  
  
.center-red {  
  color: red;  
  text-align: center;  
}
```

Код HTML

```
<p>Каскадность CSS:</p>  
<ul>  
  <li>расширение свойств;</li>  
  <li>переопределение свойств.</li>  
</ul>
```

Результат

Каскадность CSS:

- расширение свойств;
- переопределение свойств.

Хранение стилей

1. В отдельном файле с расширением **css**. Этот способ используется, если стили будут применяться к нескольким страницам. Тогда во всех этих страницах необходимо подключить созданный **css** файл.

```
<link rel="stylesheet" type="text/css" href="style.css">
```

Хранение стилей

2. Описание стилей можно хранить непосредственно в коде HTML-страницы. В этом случае область действия стилей распространяется только на текущую страницу. Стили задаются в заголовке документа с помощью парного тега **<style>**:

```
<style>
  p {
    color : blue;
  }
</style>
```

Хранение стилей

3. Описание стиля может располагаться непосредственно внутри тега элемента. Это делается с помощью атрибута **style**:

```
<p style="color: red; font-size: 12px">Стили CSS</p>
```

Этот метод нежелателен, так как он приводит к потере основных преимуществ CSS – возможности отделения кода HTML от оформления, реализации каскадности.

Приоритеты применения стилей

Если для одного HTML-документа применяется несколько способов хранения стилей, то действуют следующие правила старшинства стилей:

- сначала применяются стили умолчания браузера;
- стили умолчания браузера переопределяются прилинкованными стилями (теми стилями, которые хранятся в отдельном файле и подключаются к странице тегами `<link>`);
- прилинкованные стили переопределяются описаниями стилей в теге `<style>` текущей страницы;
- стили тега `<style>` переопределяются атрибутом **style** в любом из тегов страницы.

Типы HTML-элементов

В HTML4 практически все элементы относились к двум типам — блочные и строчные элементы.

Блочные элементы (например, `<p>`, `<ol>`, `<div>`) начинаются с новой строки и занимают всё доступное им пространство по ширине, независимо от содержимого.

Строчные (например, `<a>`, `<span>`) являются частью строки, их размеры определяются содержимым.

Типы HTML-элементов

HTML5 отказался от строгого разделения тегов на блочные и строчные, но само разделение, конечно, сохранилось. Только оно осуществляется через CSS.

Теперь у элемента есть стилевое оформление:

- блочное;
- строчное;
- строчно-блочное.

Практически любой элемент HTML-страницы может быть преобразован из строчного в блочный и наоборот для этого используется свойство **display**.

Универсальные стили элементов

Элемент HTML представляет собой прямоугольную область, в которой можно поместить некоторое содержимое (картинки, текст другие блоки):



Универсальные стили элементов

Элемент HTML представляет собой прямоугольную область, в которой можно поместить некоторое содержимое. Для элемента определяются:

- граница, которая может быть разной толщины, цвета, типа (**border**);
- внешние отступы до охватывающего элемента страницы (**margin**);
- внутренние отступы от границы до содержимого блока (**padding**);
- цвет фона или фоновая картинка (**background**).

Также для любого элемента можно задать:

- стили для шрифта (**font**);
- стили для оформления текста.

Тег <div>

Тег **<div>** выступает в качестве универсального блока, его можно рассматривать как часть HTML-страницы, но на самом деле его применение имеет смысл только в контексте CSS.

Без описания стилей – это просто новый блок, который начинается с новой строки и занимает всю ширину экрана.

Код HTML

<div>

Способы форматирования абзаца:

по центру, по правому краю, левому краю, по ширине.

</div>

Тег <div>

Пример

Код HTML

<div>

Способы форматирования абзаца:

по центру, по правому краю, левому краю, по ширине.

</div>

Код CSS

```
div {  
  width: 200px;  
  color: white;  
  background: grey;  
}
```

Результат

Способы
форматирования абзаца:
по центру, по правому
краю, левому краю, по
ширине.

Тег

Тег **** - это строковый элемент разметки, применение которого не приводит к образованию блока текста. Он может заменить теги ****, **<i>**, ****, **<sub>** и т.п.

Тег **** применяют, если необходимо каким-то особым образом отформатировать часть текста документа.

Тег

Пример

Код HTML

```
<p>  
  <span>Стиль</span> - это набор правил...  
</p>
```

Код CSS

```
span {  
  color: red;  
  font-weight: bold;  
}
```

Результат

Стиль - это набор правил....

Обобщенное свойство border

Свойство	Возможные значения
border-width задает толщину границы	• точный размер, задаваемый числом со стандартной единицей • значения: thin - тонкая линия; medium - линия средней толщины; thick - толстая линия.
border-color задает цвет границы	Цвет – значение цвета, описанное любым стандартным способом: • #RRGGBB; • color; • rgb(число, число, число); • rgba(число, число, число, прозрачность).
border-style задает стиль линии для границы	Стиль линии: • solid – обычная сплошная линия; • none – линия отсутствует, ширина равна нулю (по умолчанию); • dotted – линия в виде последовательности точек; • dashed – пунктирная линия; • ...

Обобщенное свойство border

Код CSS

```
p, span {  
    border-width: 2px;  
    border-color: green;  
    border-style: solid;  
}
```

Такой же стиль границы можно создать, используя обобщенное свойство **border**. При перечислении свойства разделяются пробелами, их порядок произволен. Обязательной характеристикой является стиль линии.

Код CSS

```
p, span {  
    border: 2px green solid;  
}
```


Описание стилей частей границы

CSS-стили позволяют оформлять отдельные части границы. Для установки верхней, правой, нижней, левой границы используются свойства

- **border-top**,
- **border-right**,
- **border-bottom**,
- **border-left**

Описание стилей частей границы

В свойствах **border-width**, **border-style** и **border-color** можно задать значения стилей для отдельных частей границ. Допускается использование от 1 до 4 значений одновременно.

Если для свойства будет указано:

- одно значение – оно будет применено ко всем сторонам элемента;
- два значения – первое будет применено к верхней и нижней границе элемента, второе – к левой и правой;
- три значения – первое будет применено к верхней границы, второе – к правой и левой границ, третье – к нижней границе.
- четыре значения – первое будет применено к верхней границе, второе – к правой, третье – к нижней, четвертое – к левой.

Описание стилей частей границы

Пример

Код CSS

```
div {  
  border-style: solid dashed;  
  border-color: green red blue;  
  border-width: 2px 1px 1px 4px;  
}
```

Результат



Обобщенное свойство background

Свойство	Возможные значения
background-color устанавливает цвет фона для элемента.	• цвет – значение цвета • transparent – устанавливает прозрачный фон.
background-image задает фоновый рисунок для элемента	• url(имя файла) – ссылка или адрес изображения; • none – фоновое изображение отсутствует.
background-repeat управляет циклическим повторением фонового рисунка	• repeat-y – размножает изображение только по вертикали; • repeat-x – размножает изображение только по горизонтали; • repeat – размножает изображение в обоих направлениях - и по горизонтали и по вертикали (по умолчанию); • no-repeat – не размножает изображение.

Обобщенное свойство **background**

С помощью обобщенного свойства **background** можно указать значения всех свойств из таблицы. При этом, если указаны:

- **цвет** - для элемента применяется цвет фона;
- **фоновый рисунок** - для элемента применяется фоновый рисунок;
- **цвет и фоновое изображение** - для элемента применяется фоновое изображение;
- **цвет, фоновое изображение и способ повторения фона** - область, в которой не отображается фон, заливается указанным цветом.

Отступы

margin - внешние отступы до охватывающего элемента страницы.

padding - внутренние отступы от границы до содержимого блока.

Можно оформить разные отступы по вертикали и горизонтали:

- **margin-top, margin-bottom, margin-right, margin-left;**
- **padding-top, padding-bottom, padding-right, padding-left.**

Отступы

Объединенное описание отступов (**margin, padding**):

- одно значение – отступ со всех сторон элемента;
- два значения – первое будет применено к верхней и нижней стороне элемента, второе – к левой и правой;
- три значения – первое будет применено к верхней стороне, второе – к правой и левой границ, третье – к нижней стороне.
- четыре значения – первое будет применено к верхней границе, второе – к правой, третье – к нижней, четвертое – к левой.

Обобщенное свойство font

Свойство	Возможные значения
font-style наклон шрифта	• normal - обычный прямой шрифт, без наклона (по умолчанию); • italic - курсивный шрифт - наклонный шрифт, имитирующий рукописный; • oblique - наклонный шрифт, курсив.
font-weight насыщенность шрифта	• normal - шрифт стандартной толщины (по умолчанию); • bold - стандартный полужирный шрифт.
font-size размер шрифта	• <i>именованный размер</i>: xx-small, x-small, small, medium, large, x-large, xx-large. • <i>точный размер</i>, задаваемый числом с единицей измерения: - стандартные единицы, применяемой для стилей (px и пр.); - em - ширина буквы m в текущем шрифте; - ex - высота буквы x в текущем шрифте. • <i>размер в процентах</i>, задаваемый числом и знаком процента.
line-height размер межстрочного интервала	• normal - высота межстрочного интервала зависит от размера и вида шрифта, вычисляется автоматически (по умолчанию); • <i>число-множитель</i> - положительное число, на которое умножается размер текущего шрифта; • <i>число с единицей измерения</i>; • <i>число%</i> - процент будет вычисляться от размера текущего шрифта.
font-family тип шрифта	• <i>имя шрифта</i> - имя или имена шрифтов через запятую; • <i>семейство</i> - одно из семейств шрифтов.

Обобщенное свойство font

Для описания стиля шрифта можно использовать обобщенное свойство **font**. Его значения могут быть любыми из таблицы.

При перечислении они разделяются пробелами, их порядок должен быть таким же, как в таблице, при этом обязательными являются только значения свойств **font-family** и **font-size**.

Значения **line-height** можно указать только в паре с **font-size**, разделив их символом «/».

Обобщенное свойство font

Пример

Код CSS

```
p {  
  font-weight: bold;  
  font-style: italic;  
  font-size: 14px;  
  line-height: 2;  
  font-family: arial;  
}
```

Код CSS

```
p {  
  font: bold italic 14px/2 arial;  
}
```


Стили для оформления текста

Свойство	Возможные значения
color цвет текста	Стандартные способы описания цвета
text-decoration способы оформления текста	• none - отменяет все оформительские приемы; • underline - текст будет отображаться с нижним подчеркиванием; • overline - текст будет отображаться с верхним подчеркиванием; • line-through - текст будет отображаться зачеркнутым.
letter-spacing интервал между буквами	• <i>точный размер</i>, задаваемый числом с единицей измерения: - стандартные единицы, применяемой для стилей (px и пр.); - em - ширина буквы m в текущем шрифте. • <i>размер в процентах</i>, задаваемый числом и знаком процента.
word-spacing интервал между словами	• <i>точный размер</i>, задаваемый числом с единицей измерения: - стандартные единицы, применяемой для стилей (px и пр.); - em - ширина буквы m в текущем шрифте. • <i>размер в процентах</i>, задаваемый числом и знаком процента.

Блочные элементы

Блочный элемент - это элемент, который занимает всю доступную ширину и всегда начинается с новой строки. Высота блока зависит от его содержимого. У этих элементов в CSS стилях свойство **display** задано как:

- **block** - элемент отображается как прямоугольник, ширина которого занимает 100% ширины охватывающего элемента.
- **list-item** - элемент отражается, как элемент списка;
- **table** - элемент отображается как таблица.

Для некоторых тегов эти значения **display** установлены по умолчанию (например, `<h1>...<h6>`, `<p>`, `<ol>`, `<ul>`, `<div>`), поэтому они уже обладают свойствами блочных элементов.

Блочные элементы

Для блочных элементов можно:

- установить ширину блока (**width**);
- установить высоту блока (**height**);
- выровнять по центру страницы (**margin: auto**);
- управлять отображением содержимого блока (**overflow**);
- управлять обтеканием блока (**float, clear**);
- использовать свойства текста для блочных элементов (**text-align, text-align-last, text-indent, text-transform**);
- преобразовать в строчный элемент.

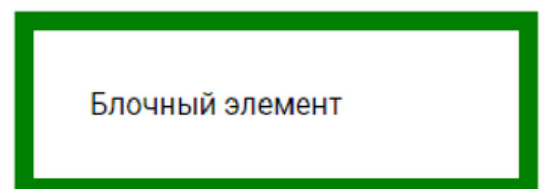
Ширина блочного элемента

Блочный элемент по умолчанию занимает всю доступную ему ширину.

Для изменения ширины блока используется свойство **width**, которое устанавливает ширину содержимого элемента (на рисунке это область «Блочный элемент»).

Ширина блочного элемента определяется сложением значений:

- **margin-left**;
- **border-left**;
- **padding-left**;
- **width**;
- **padding-right**;
- **border-right**;
- **margin-right**.



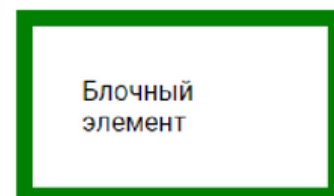
Блочный элемент

Изменение алгоритма расчета ширины

Чтобы изменить алгоритм расчета ширины блока используется свойство **box-sizing** со значением **border-box**.

Тогда свойство **width** включает:

- ширину содержимого;
- внутренние отступы (**padding**)
- толщину границы.



Блочный элемент

Выравнивание по центру страницы

По умолчанию блочный элемент прижат к левой границе окна браузера или охватывающего элемента.

Для того, чтобы его "выровнять" по центру, необходимо в качестве значения правого и левого отступа внешней границы (свойство **margin**) указать значение **auto**.

При этом у элемента должна быть обязательно задана ширина (свойство **width**).

Код CSS

```
.block {  
  width: 300px;  
  margin: 5px auto;  
}
```

Код CSS

```
.block {  
  width: 300px;  
  margin: auto;  
}
```

Управление отображением

Управление отображением содержимого блочного элемента, если оно не помещается в область заданных размеров, осуществляется с помощью свойства **overflow**.

- **visible** – отображает все содержание элемента, даже за пределами установленной ширины и высоты (по умолчанию);
- **hidden** – отображает только область внутри элемента, остальное будет скрыто;
- **scroll** – всегда добавляются полосы прокрутки;
- **auto** – полосы прокрутки добавляются при необходимости.

Управление отображением

Пример

Код HTML

```
<div class="content">
  <div class="block">
    блок 1
  </div>
  <div class="block">
    блок 2
  </div>
</div>
```

Результат:



Код CSS

```
.content {
  width: 200px;
  height: 50px;
  margin: 5px;
  border: solid thin red;
}
.block {
  width: 300px;
  margin: 5px;
  padding: 5px;
  border: dashed thin green;
}
```

Управление отображением

Пример

Код HTML

```
<div class="content">
  <div class="block">
    блок 1
  </div>
  <div class="block">
    блок 2
  </div>
</div>
```

Результат:



Код CSS

```
.content {
  overflow: hidden;
  width: 200px;
  height: 50px;
  margin: 5px;
  border: solid thin red;
}
.block {
  width: 300px;
  margin: 5px;
  padding: 5px;
  border: dashed thin green;
}
```

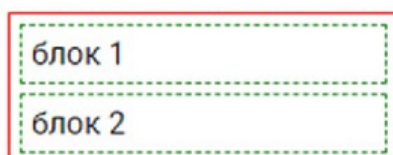
Управление отображением

Пример

Код HTML

```
<div class="content">
  <div class="block">
    блок 1
  </div>
  <div class="block">
    блок 2
  </div>
</div>
```

Результат:



Код CSS

```
.content {
  overflow: hidden;
  /*height: 50px;*/
  margin: 5px;
  border: solid thin red;
}
.block {
  /* width: 300px; */
  margin: 5px;
  padding: 5px;
  border: dashed thin green;
}
```


Обтекание блочного элемента(float)

Свойство **float** определяет, по какой стороне будет выравниваться элемент, при этом остальные элементы будут обтекать его с других сторон.

Код HTML

```
<div class="content">
  <div class="block">
    image
  </div>
  Свойство float ....
</div>
```

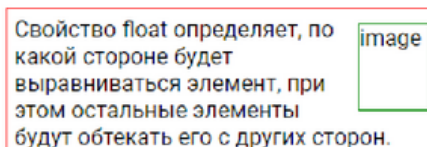
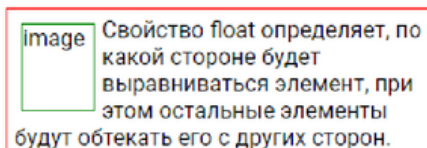
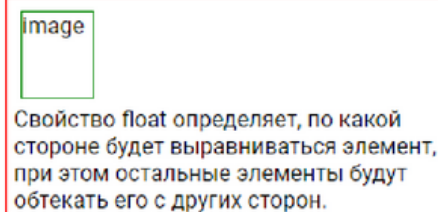
Код CSS

```
.content {
  width :300px;
  padding: 5px;
  border: solid thin red;
}
.block {
  float: /* будем задавать
значение */
  width: 50px;
  height: 60px;
  margin: 5px;
  border: solid thin green;
```

Обтекание блочного элемента(float)

Возможные значения свойства **float**:

- **none** - обтекание элемента не задается, элемент выводится на странице как обычно (установлен по умолчанию);
- **left** - выравнивает элемент по левому краю, а все остальные элементы, например, текст, обтекают его по правой стороне;
- **right** - выравнивает элемент по правому краю, а все остальные элементы обтекают его по левой стороне.



Обтекание блочного элемента(float)

Пример (для вложенных блоков).

- **float: none;**
- **float: left;**
- **float: left;**
overflow: hidden;
- **float: right; overflow: hidden;**



Обтекание блочного элемента(clear)

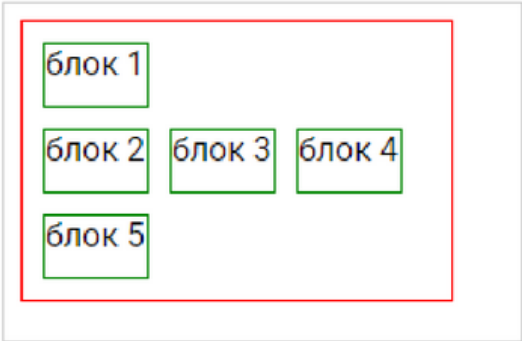
Свойство **clear** устанавливает, с какой стороны элемента запрещено его обтекание другими элементами. Если задано обтекание элемента с помощью свойства **float**, то **clear** отменяет его действие для указанных сторон.

Возможные значения:

- **none** - отменяет действие свойства **clear**, при этом обтекание элемента происходит, как задано с помощью свойства **float**;
- **both** - отменяет обтекание элемента одновременно с правого и левого края;
- **left** - отменяет обтекание с левого края элемента, при этом все другие элементы на этой стороне будут опущены вниз, и располагаться под текущим элементом;
- **right** - отменяет обтекание с правой стороны элемента.

Обтекание блочного элемента(clear)

Пример:



Для всех блоков установлено левое обтекание (**float: left**)

Для блока 2 отменено обтекание **left** (теперь для этого блока действует свойство **float: none**).

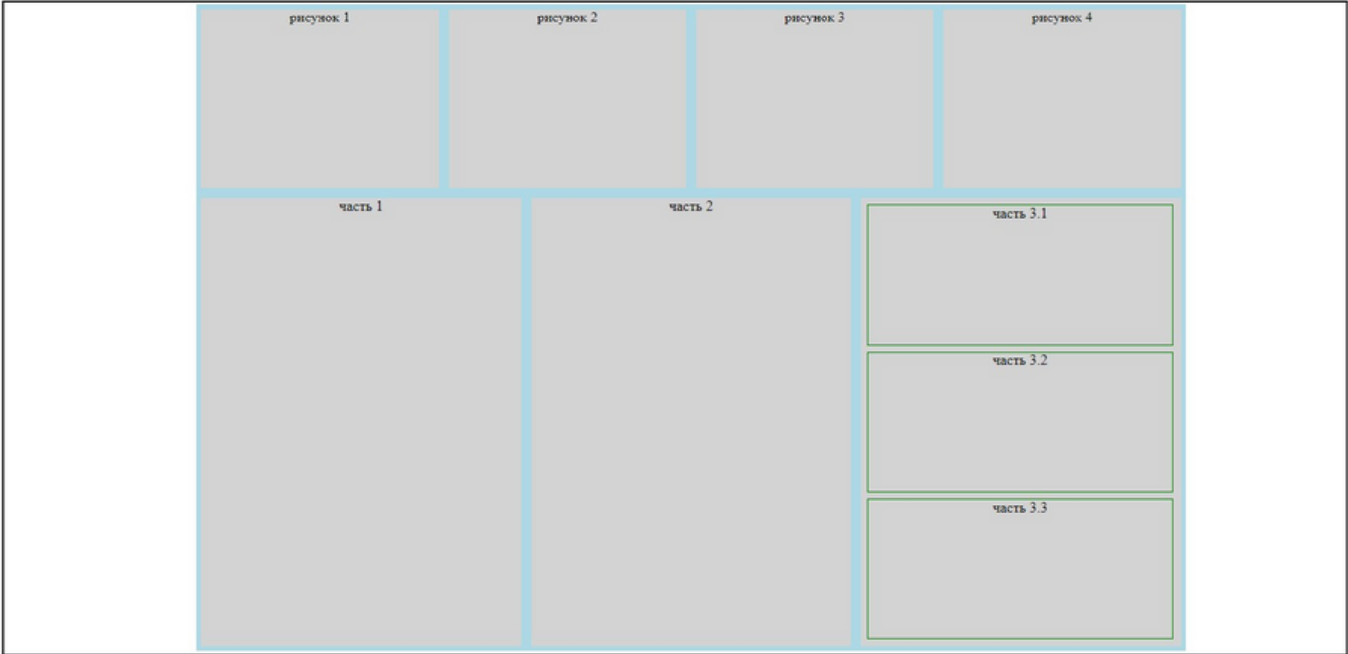
Поэтому отображение блока 2 началось с новой строки.

Стили текста

Свойство	Возможные значения
text-align выравнивает содержимое блочного элемента относительно границ блока	left - выравнивание текста по левой границе. • right - выравнивание по правой границе. • center - выравнивание по центру. • justify - выравнивание по ширине.
text-align-last выравнивание последней строки текста, если свойство text-align установлено как justify	• left - строка выравнивается по левой границе. • right - строка выравнивается по правой границе. • center - строка выравнивается по центру. • justify - строка выравнивается по ширине
text-indent размер отступа первой строки текста	• <i>точный размер</i> (можно отрицательный) • <i>размер в процентах</i>
text-transform управляет регистром отображения символов	• capitalize - первый символ каждого слова в предложении будет заглавным, остальные символы свой вид не меняют. • lowercase - все символы текста становятся строчными. • uppercase - все символы текста становятся прописными. • none - регистр символов не изменяется.

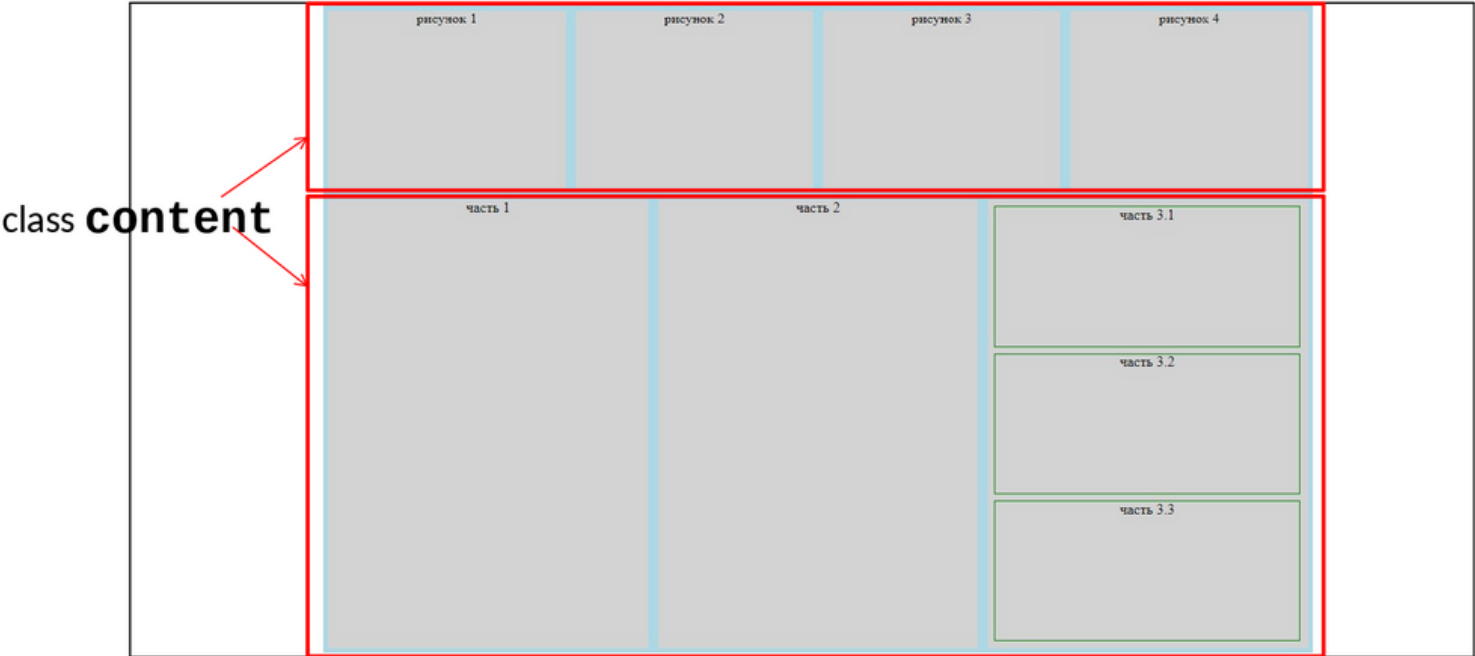
Блочный элемент, пример

Пример. С помощью блочных элементов создать страницу следующей структуры:



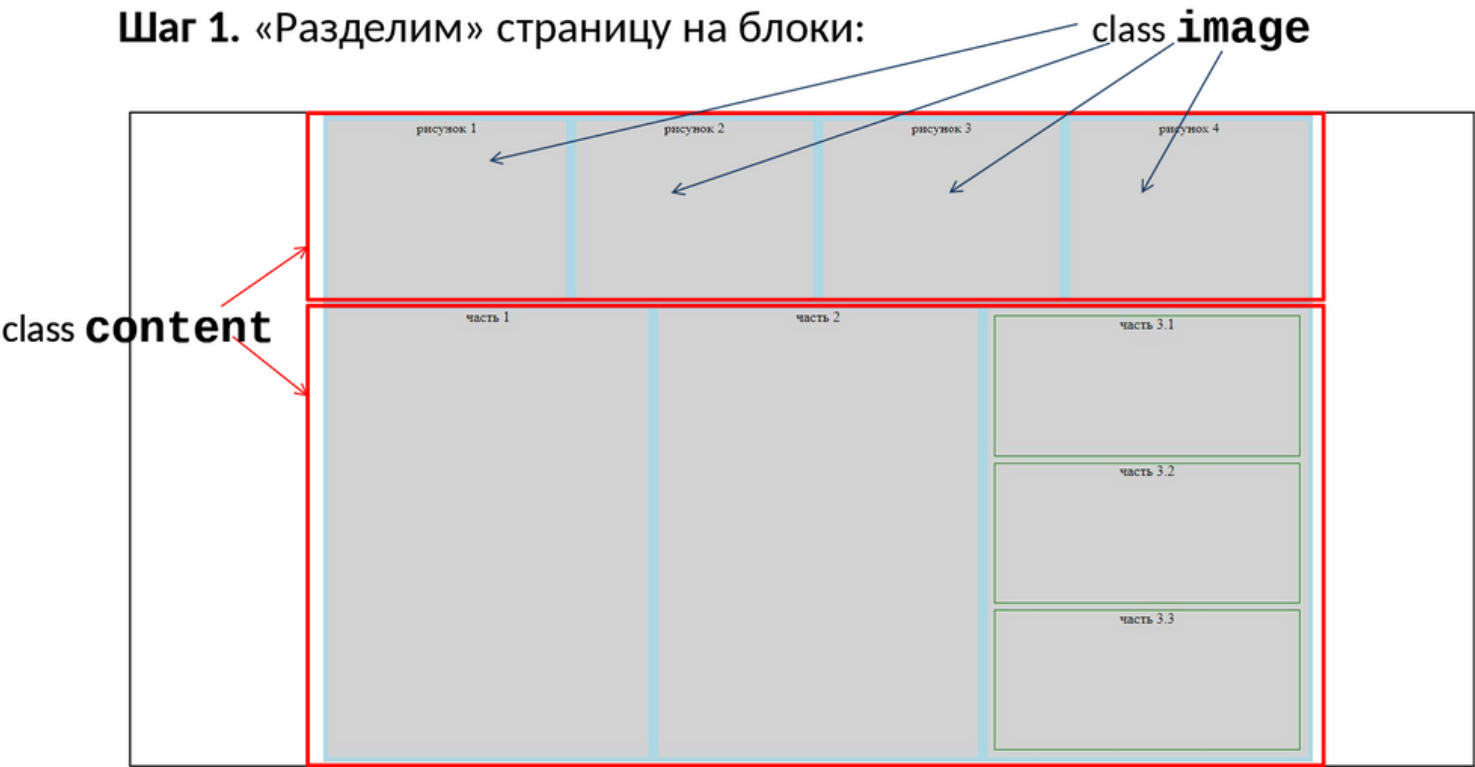
Блочный элемент, пример

Шаг 1. «Разделим» страницу на блоки:



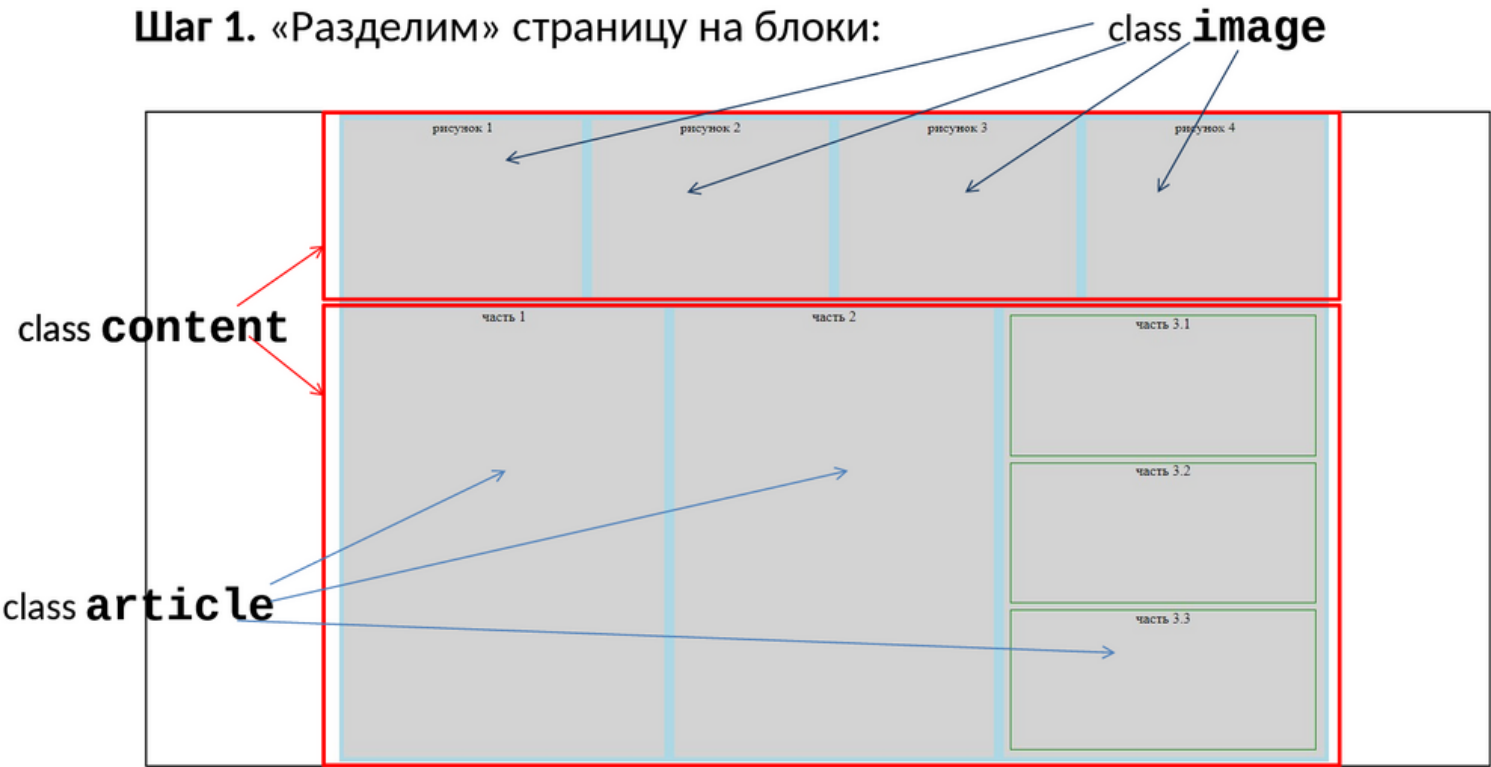
Блочный элемент, пример

Шаг 1. «Разделим» страницу на блоки:



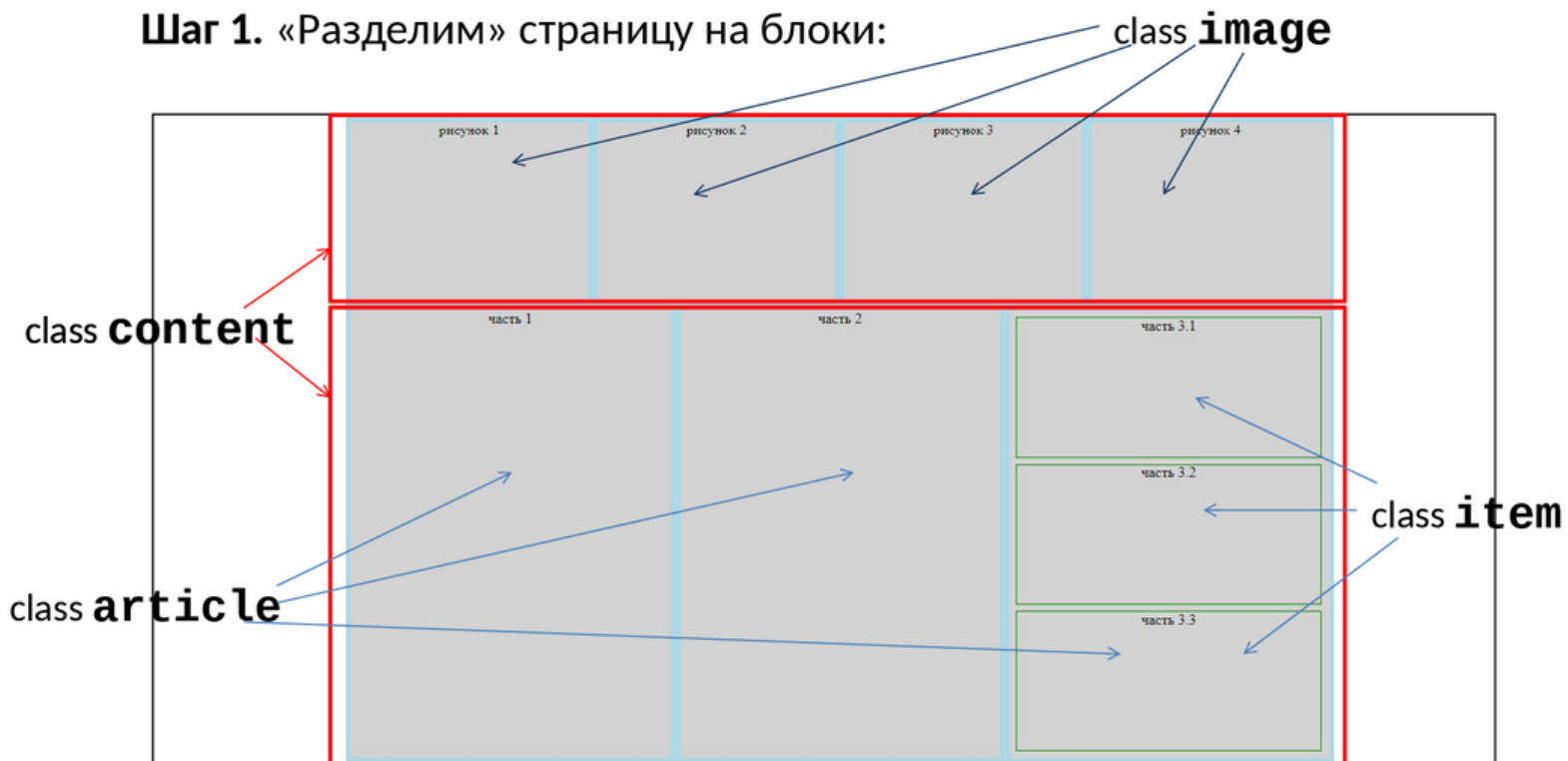
Блочный элемент, пример

Шаг 1. «Разделим» страницу на блоки:



Блочный элемент, пример

Шаг 1. «Разделим» страницу на блоки:



Блочный элемент, пример

Шаг 2. Реализуем соответствующий HTML-код

```
<div>
  <div>рисунок 1</div>
  <div>рисунок 2</div>
  <div>рисунок 3</div>
  <div>рисунок 4</div>
</div>
<div>
  <div>часть 1</div>
  <div>часть 2</div>
  <div>
    <div>часть 3.1</div>
    <div>часть 3.2</div>
    <div>часть 3.3</div>
  </div>
</div>
```

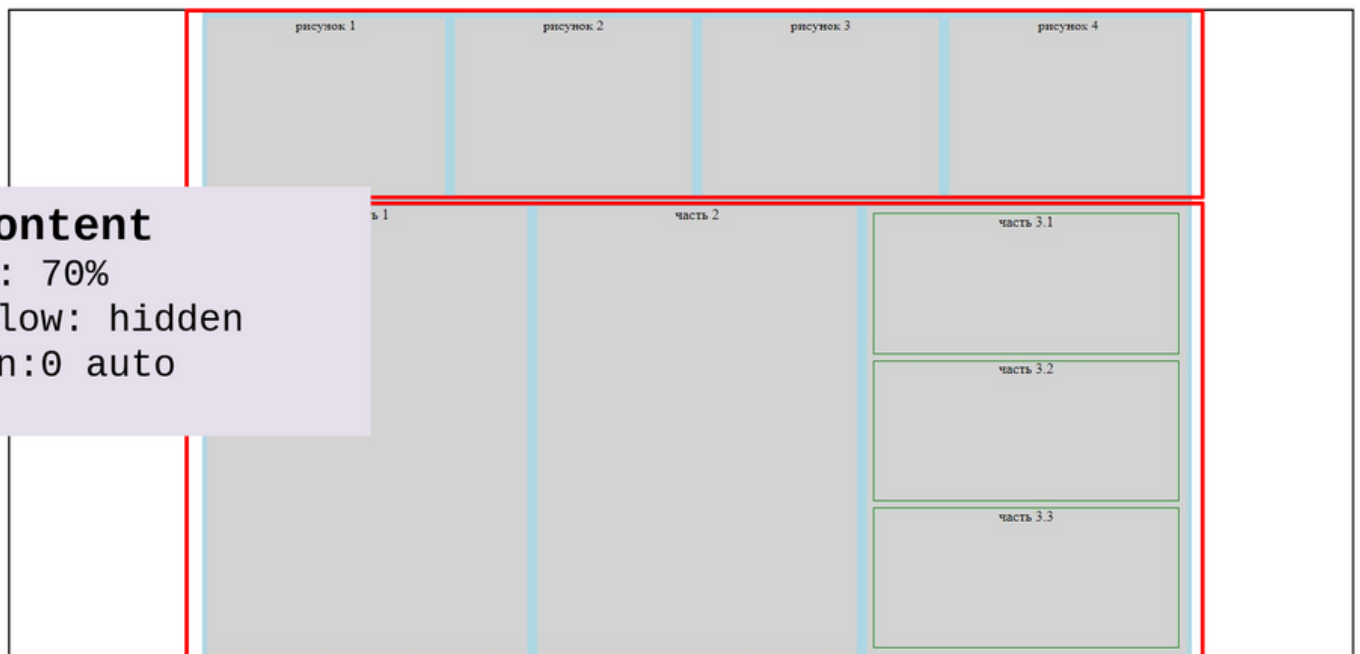
Блочный элемент, пример

Шаг 3. Относим элементы к классам

```
<div class="content">
  <div class="image">рисунок 1</div>
  <div class="image">рисунок 2</div>
  <div class="image">рисунок 3</div>
  <div class="image">рисунок 4</div>
</div>
<div class="content">
  <div class="article">часть 1</div>
  <div class="article">часть 2</div>
  <div class="article">
    <div class="item">часть 3.1</div>
    <div class="item">часть 3.2</div>
    <div class="item">часть 3.3</div>
  </div>
</div>
```

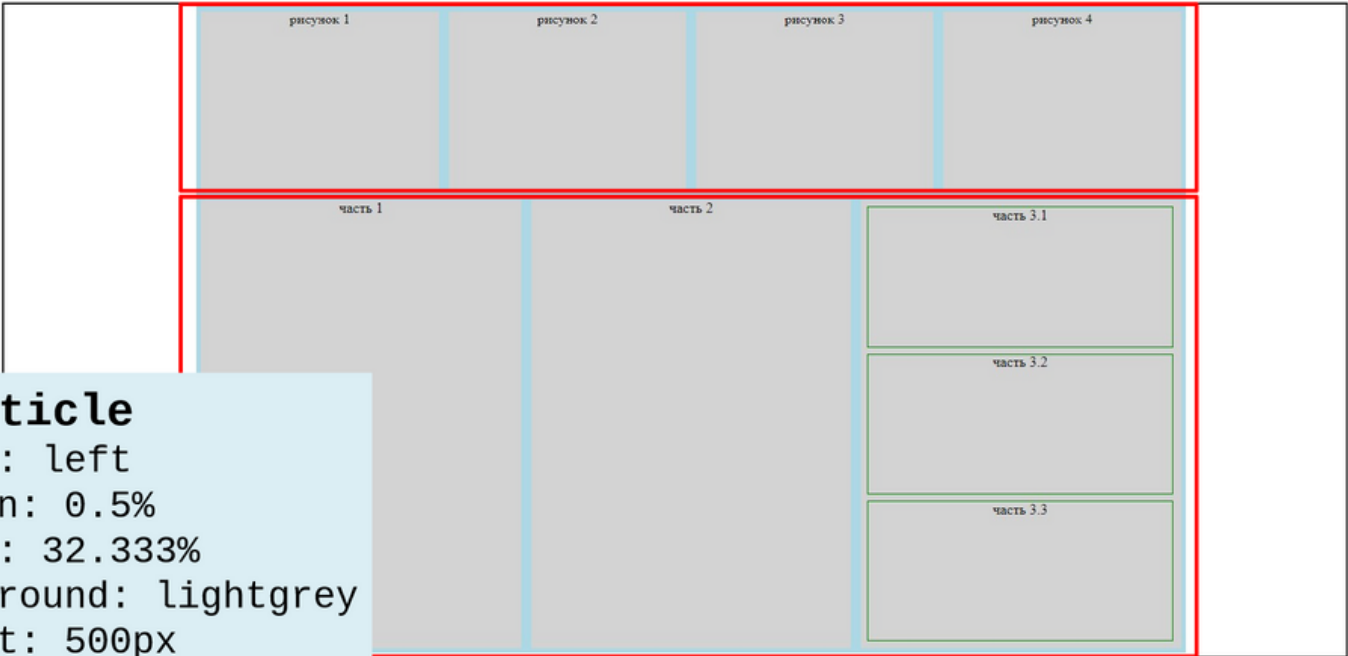
Блочный элемент, пример

Шаг 4. Описываем стили для каждого класса.



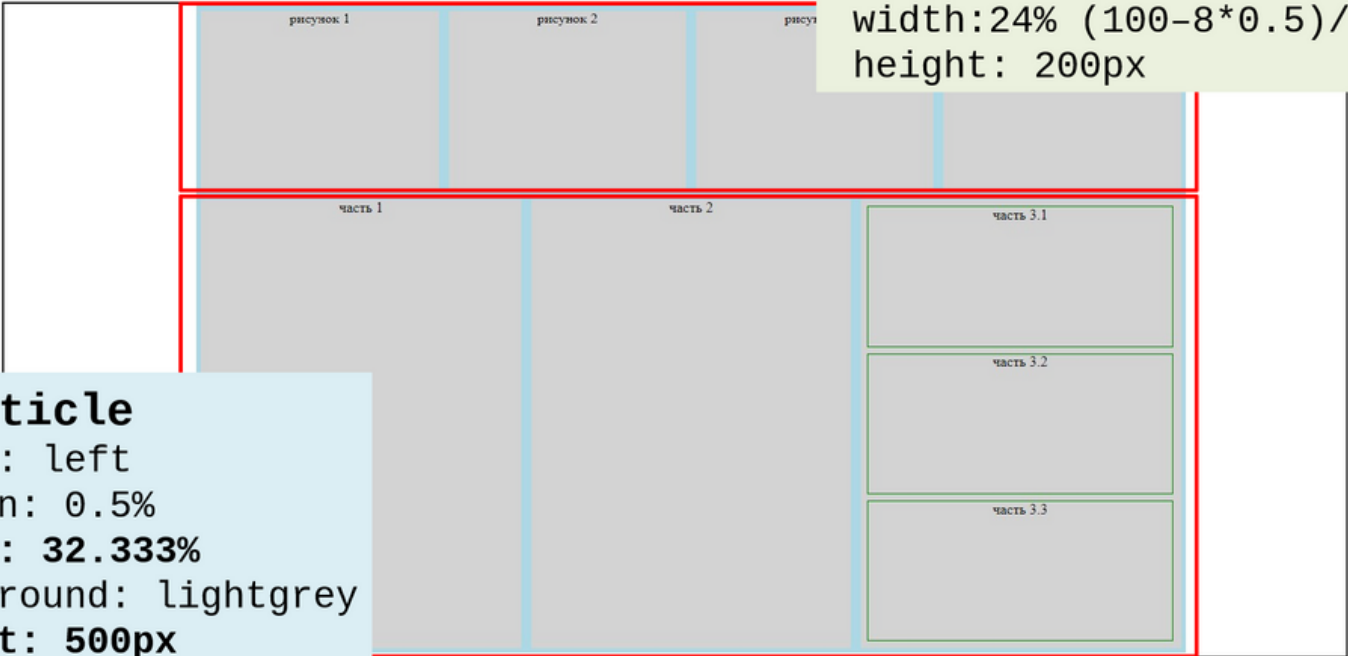
Блочный элемент, пример

Шаг 4. Описываем стили для каждого класса.



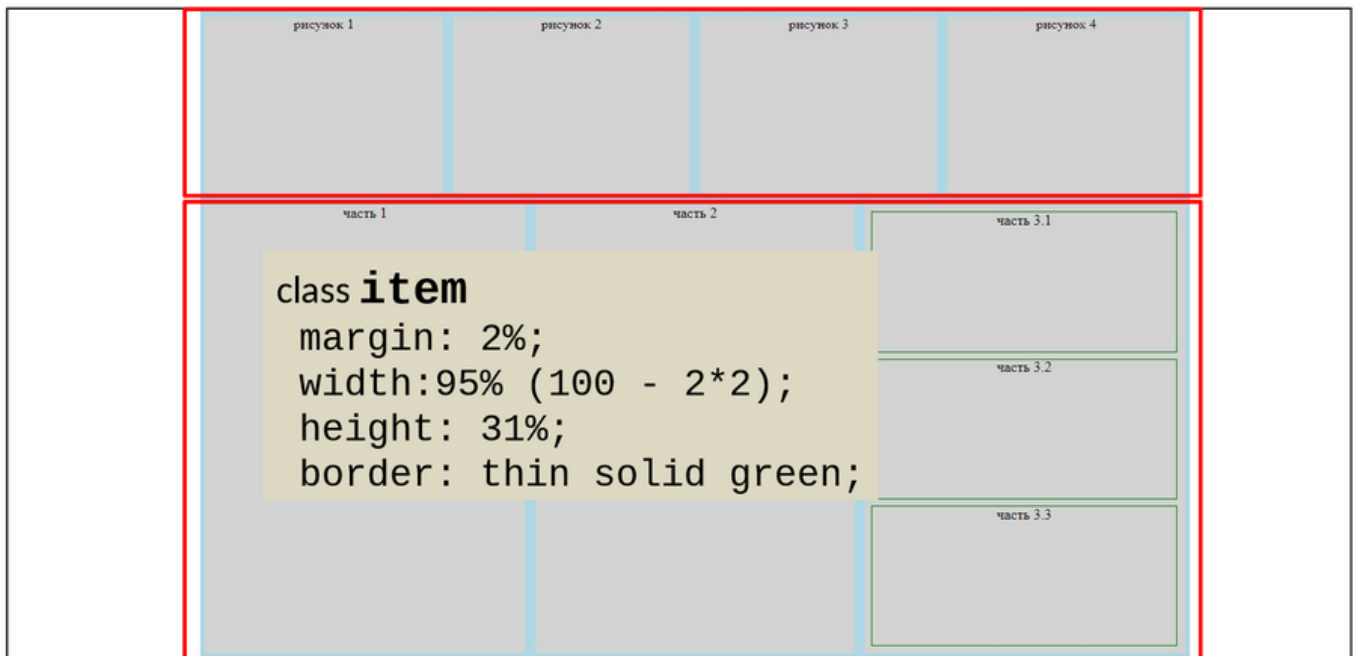
Блочный элемент, пример

Шаг 4. Описываем стили для каждого класса.



Блочный элемент, пример

Шаг 4. Описываем стили для каждого класса.



Блочный элемент, пример

Шаг 4. Если необходимо – добавляем классы в HTML.

```
<div class="content">
  <div class="article image">рисунок 1</div>
  <div class="article image">рисунок 2</div>
  <div class="article image">рисунок 3</div>
  <div class="article image">рисунок 4</div>
</div>
<div class="content">
  <div class="article">часть 1</div>
  <div class="article">часть 2</div>
  <div class="article">
    <div class="item">часть 3.1</div>
    <div class="item">часть 3.2</div>
    <div class="item">часть 3.3</div>
  </div>
</div>
```

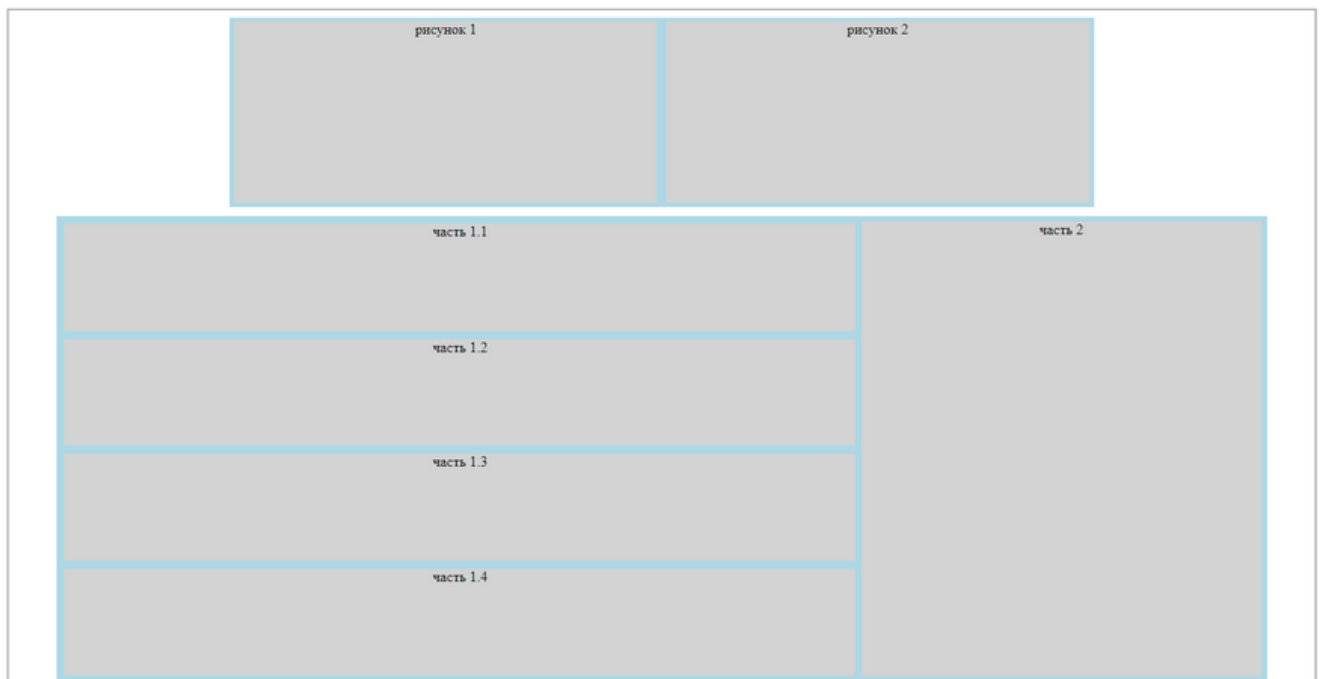
Блочный элемент, пример

Шаг 5. Описываем стили для каждого класса.

```
.content {  
  width: 70%;  
  overflow: hidden;  
  margin: 0 auto;  
  background: lightblue;  
  text-align: center;  
}  
.article {  
  float: left;  
  width: 32.333%;  
  margin: 0.5%;  
  background: lightgrey;  
  height: 500px;  
}  
.image {  
  width: 24%;  
  height: 200px;  
}  
.item {  
  width: 95%;  
  float: left;  
  margin: 2%;  
  height: 31%;  
  border: thin solid green;  
}
```

Блочный элемент

Задание 4. Написать HTML и CSS коды для следующей страницы:



Строчный элемент

Строчный элемент - это элемент, который является непосредственной частью строки, у него значение свойства **display** установлено как **inline**.

Для некоторых тегов это значение задано по умолчанию (например, `<span>`, `<a>`, `<q>`, `<code>`), в основном они используются для изменения вида текста и его фрагментов.

Строчный элемент

Особенности строчных элементов

- свойства **width**, **height** неприменимы;
- размер элемента вычисляется как сумма его содержимого и значений **margin**, **border** и **padding** ;
- строчный элемент переносится на новую строку, для отмены переноса используется свойство **white-space**;
- в коде HTML переход на следующую строку, а также размещение тегов на отдельных строках, воспринимается браузером как пробел;
- строчный элемент можно выравнивать по вертикали с помощью свойства **vertical-align**;
- строчный элемент можно преобразовать в блочный.

Размер строчного элемента

Размер строчного элемента определяется его содержимым, их нельзя изменять с помощью **width** и **height**. Если в стилях, описывающих строчный элемент, встречаются эти свойства, то они просто игнорируются.

Высота элемента включает:

- **border-top**;
- **padding-top**;
- высоты содержимого;
- **padding-bottom**;
- **border-bottom**.

Ширина элемента включает:

- **margin-left**;
- **border-left**;
- **padding-left**;
- ширины содержимого;
- **padding-right**;
- **border-right**;
- **margin-right**.

Перенос содержимого строчного элемента

Если строчный элемент не помещается в выделенный размер по ширине, то он переносится на следующую строку.

Если строчный элемент включает текст, то перенос текста происходит на месте пробела (или другого разделителя слова, например "-").

Строчный элемент может быть разделен на несколько частей.

Ширина строчного элемента складывается из: **margin-left**, **border-left**, **padding-left**, ширины содержимого, **padding-right**, **border-right**, **margin-right**.

Перенос содержимого строчного элемента

Чтобы запретить перенос текста внутри элемента, используется свойство **white-space** со значением **nowrap**.

Если строковый элемент не помещается в отведенную область, то он "обрезается".

Ширина строчного элемента складывается из:
margin-left, border-left, padding-left, ширины содержимого, padding-right, border-right, margin-right.

Ширина строчного элемента складывается из:
margin-left, border-left, padding-left, ширины содержимого, padding-right, border-right, margin-right.

Размещение строчных элементов в коде

В коде HTML переход на следующую строку, а также размещение тегов на отдельных строках, воспринимается браузером как пробел.

```
<p>  
Теги для описания заголовков:  
<span>h1,</span>  
<span>h2,</span>  
<span>h3,</span>  
<span>h4,</span>  
<span>h5,</span>  
<span>h6,</span>  
</p>
```

Теги для описания заголовков: h1, h2, h3, h4, h5, h6.

Размещение строчных элементов в коде

В коде HTML переход на следующую строку, а также размещение тегов на отдельных строках, воспринимается браузером как пробел.

```
<p>
```

Теги для описания заголовков:

```
<span>h1,</span><span>h2,</span><span>h3,</span>
n>
  <span>h4,</span>
  <span>h5,</span>
  <span>h6,</span>
</p>
```

Теги для описания заголовков: h1,h2,h3, h4, h5, h6.

Вертикальное выравнивание

Свойство **vertical-align** выравнивает строчные элементы относительно друг друга по вертикали.

Код HTML

```
<p>
  
  Chrome
  <span>Google</span>
</p>
```

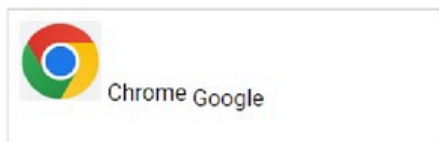
Код CSS

```
img {
  vertical-align: __ ;
}
span {
  vertical-align: __;
}
```

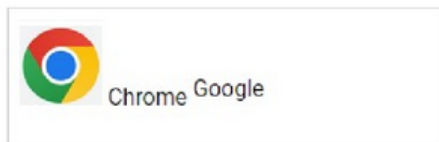
Вертикальное выравнивание

Свойство **vertical-align** может принимать следующие значения:

- **sub** - отображает элемент со сдвигом вниз, размер не меняется (свойство устанавливается для тега **span**, для тега **img** свойство не установлено):



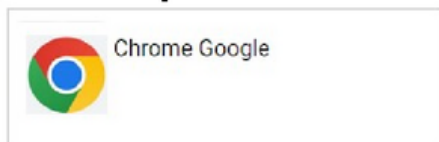
- **super** - отображает элемент со сдвигом вверх, размер при этом не меняется (свойство устанавливается для тега **span**, для тега **img** свойство не установлено):



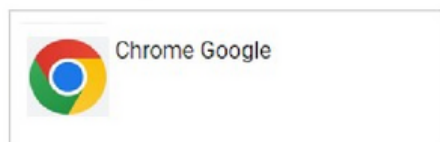
Вертикальное выравнивание

Свойство **vertical-align** может принимать следующие значения:

- **text-top** - верхняя граница элемента будет выравниваться по самому высокому текстовому элементу строки (свойство устанавливается для тега **img**, для тега **span** свойство не установлено):



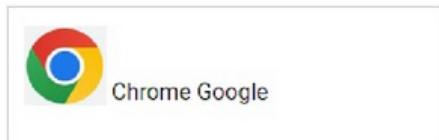
- **text-bottom** - нижняя граница элемента будет выравниваться по самому нижнему краю текущей строки (свойство устанавливается для тега **img**, для тега **span** свойство не установлено):



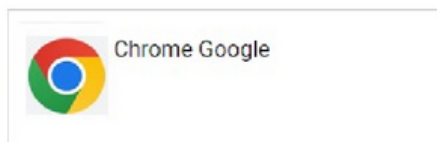
Вертикальное выравнивание

Свойство **vertical-align** может принимать следующие значения:

- **bottom** - выравнивает основание текущего элемента по нижней части элемента строки, расположенного ниже всех (свойство устанавливается для тега **img**, для тега **span** свойство не установлено):



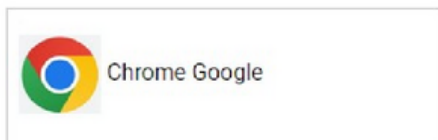
- **top** - выравнивание верхнего края элемента по верху самого высокого элемента строки (свойство устанавливается для тега **img**, для тега **span** свойство не установлено):



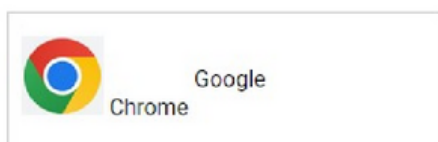
Вертикальное выравнивание

Свойство **vertical-align** может принимать следующие значения:

- **middle** - выравнивание средней точки элемента по базовой линии родителя плюс половина высоты родительского элемента (свойство устанавливается для тега **img**, для тега **span** свойство не установлено):



- **точный размер (размер в процентах)** - поднимает (положительное число) или опускает (отрицательное число) элемент на указанную величину относительно базовой линии (свойство устанавливается для тега **span**, для тега **img** свойство не установлено):



Строчно-блочные элементы

Строчно-блочный элемент объединяет свойства блочных и строчных элементов. У него значение свойства **display** имеет значение **inline-block**.

Для некоторых тегов это значение установлено по умолчанию (например, `<button>`, `<input>`, `<textarea>`, `<select>`).

Строчно-блочные элементы

Как у блочных элементов:

- высота и ширина элемента вычисляется браузером автоматически, исходя из содержимого блока;
- размеры содержимого можно задать свойствами **width** и **height**
- ширина блока определяется суммой значений **width**, **margin**, **border** и **padding**;
- высота блока определяется суммой значений **height**, **margin**, **border** и **padding**;
- управление отображением содержимого элемента, если оно не помещается в область заданных размеров, осуществляется с помощью свойства **overflow**.

Строчно-блочные элементы

Отличия от блочных элементов:

- Невозможно выровнять элемент по центру с помощью свойства **margin: auto**.
- отличается результат применения свойства **float**, которое определяет, по какой стороне будет выравниваться блочный элемент, при этом остальные элементы будут обтекать его с других сторон. Для значений **left** и **right** поведение строчно-блочного элемента совпадает с блочным, но если его не указать или задать ему значение **none**, элемент получит свойства строчных элементов.

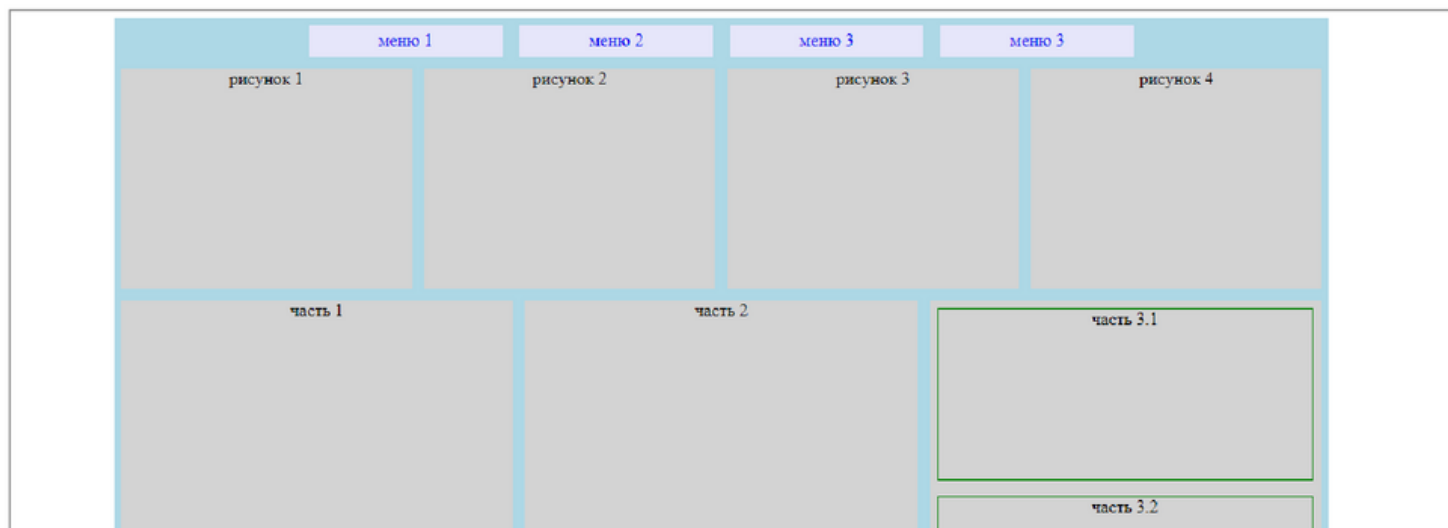
Строчно-блочные элементы

Как у строчных элементов:

- центрирование строчно-блочного элемента осуществляется с помощью свойства **text-align**.
- строчный элемент переносится на новую строку, для отмены переноса используется свойство **white-space**;
- в коде HTML переход на следующую строку, а также размещение тегов на отдельных строках, воспринимается браузером как пробел;
- строчный элемент можно выравнивать по вертикали с помощью свойства **vertical-align**.

Строчно-блочный элемент, пример

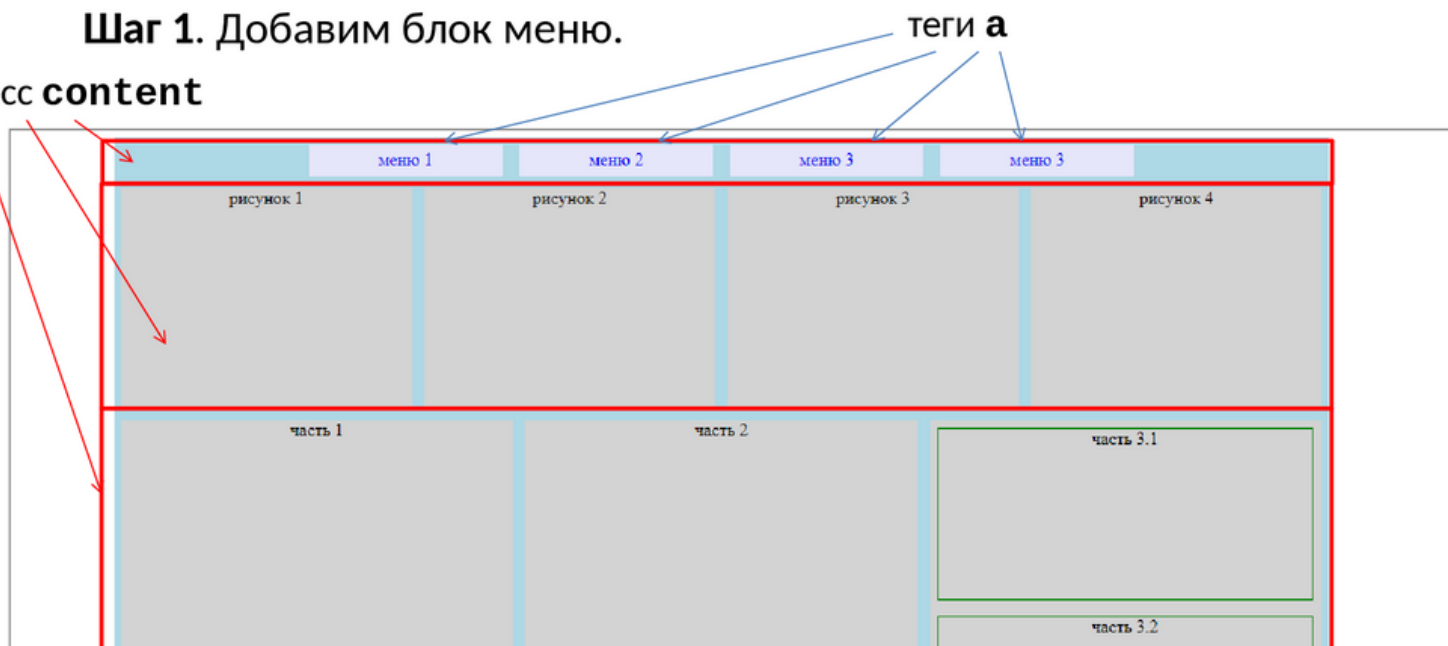
Пример. С помощью строчно-блочных элементов добавить на страницу раздел меню.



Строчно-блочный элемент, пример

Шаг 1. Добавим блок меню.

класс **content**



Строчно-блочный элемент, пример

Шаг 2. Реализуем соответствующий HTML-код

```
<div>
  <a href="#"> меню 1 </a>
  <a href="#"> меню 2 </a>
  <a href="#"> меню 3 </a>
  <a href="#"> меню 3 </a>
</div>
<div class="content">
  <div class="article image">рисунок 1</div>
  ...
</div>
<div class="content">
  <div class="article">часть 1</div>
  ...
</div>
```

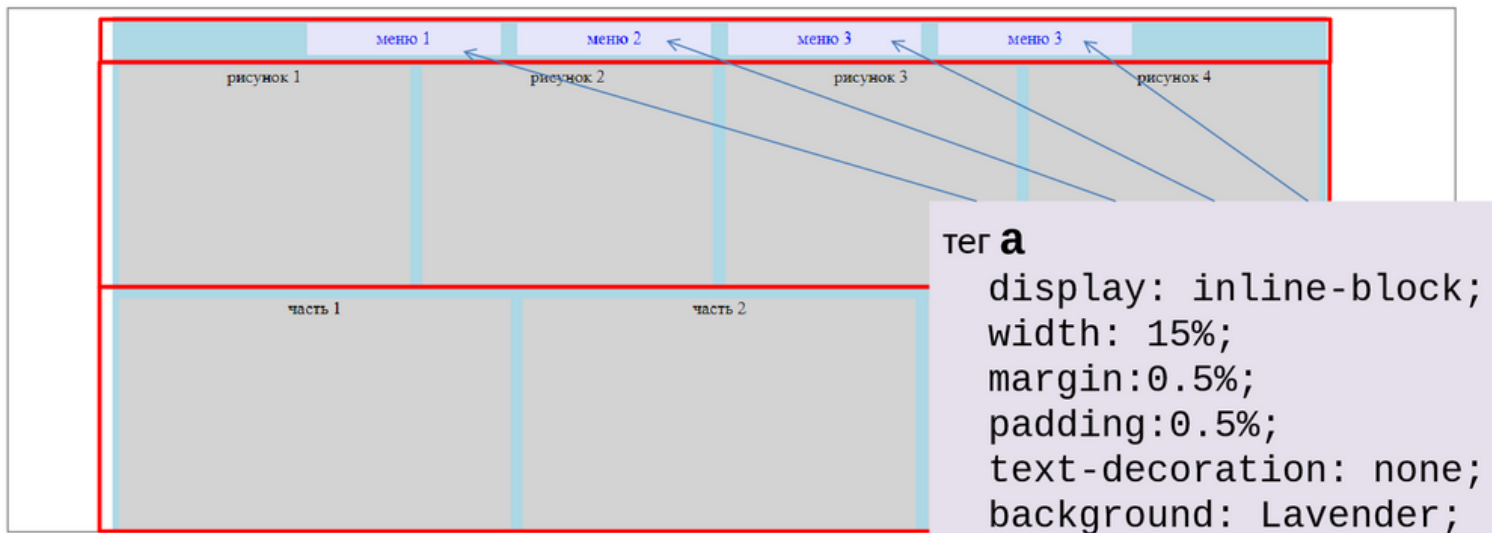
Строчно-блочный элемент, пример

Шаг 3. Добавляем классы к коду.

```
<div class="content">
  <a href="#"> меню 1 </a>
  <a href="#"> меню 2 </a>
  <a href="#"> меню 3 </a>
  <a href="#"> меню 3 </a>
</div>
<div class="content">
  <div class="article image">рисунок 1</div>
  ...
</div>
<div class="content">
  <div class="article">часть 1</div>
  ...
</div>
```


Строчно-блочный элемент, пример

Шаг 4. Описываем стили для каждого класса и тегов.



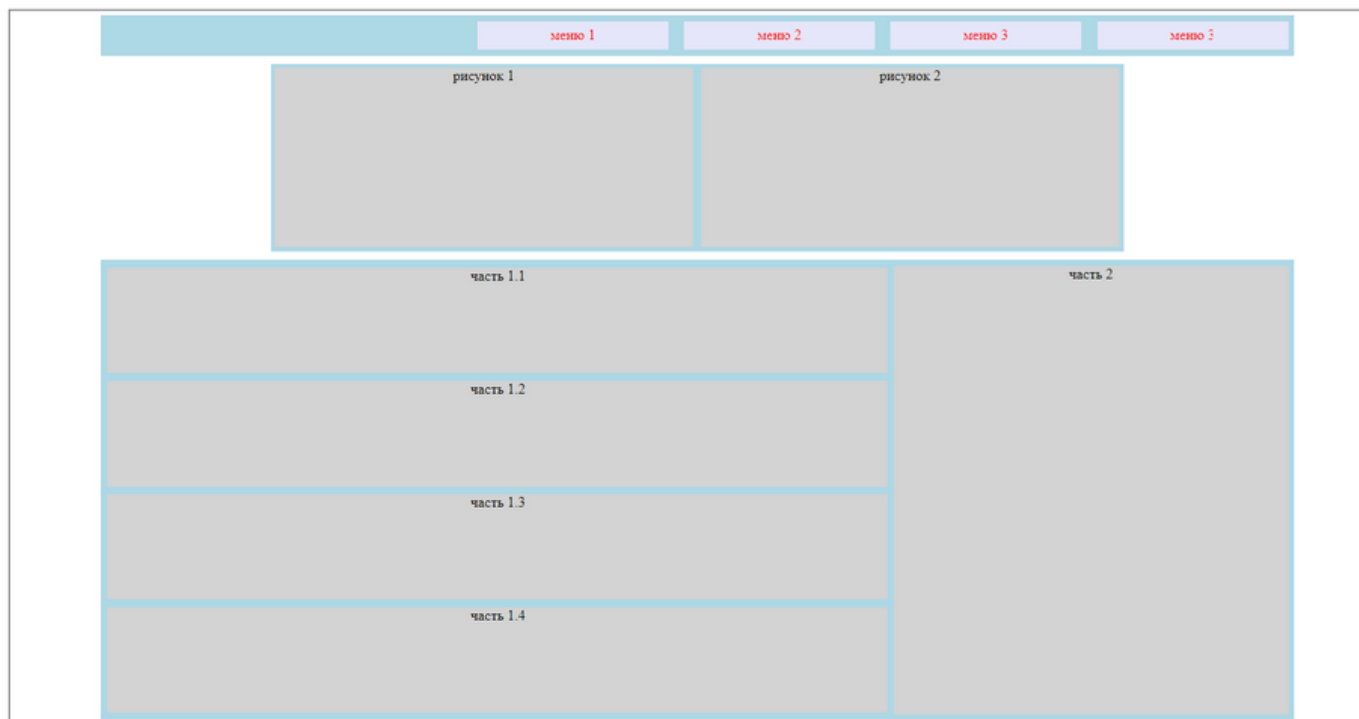
Строчно-блочный элемент, пример

Шаг 4. Описываем стили для каждого класса и тега.

```
a {
  display: inline-block;
  width: 15%;
  margin: 0.5%;
  padding: 0.5%;
  text-decoration: none;
  background: Lavender;
}
```

Строчно-блочный элемент

Задание 5. Добавить раздел меню к странице.



Спасибо за внимание!